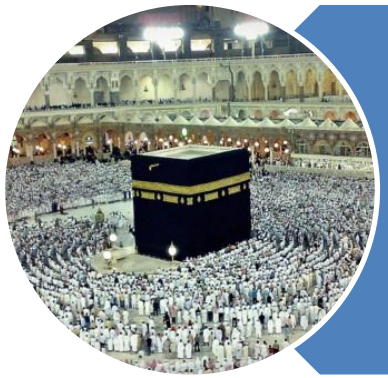
Graph Material

Gr
ap
h

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي اَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. (Quran 20 : 25-28)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Graph

Introduction

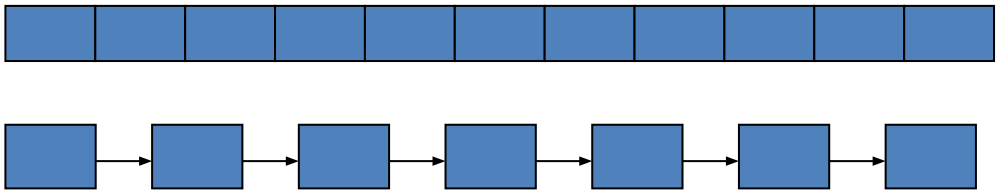
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

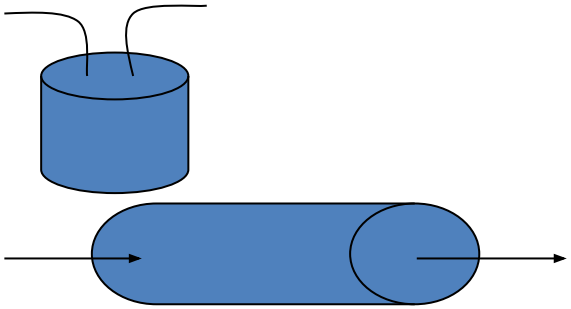
Gr
ap
h

Graph - Introduction

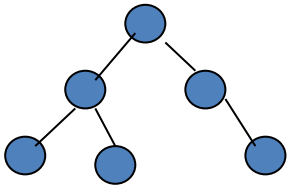
- Arrays
- Linked List



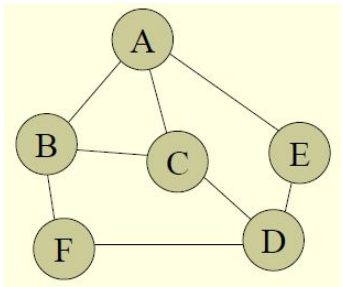
- Stacks
- Queues



- Tree



- **Graph**

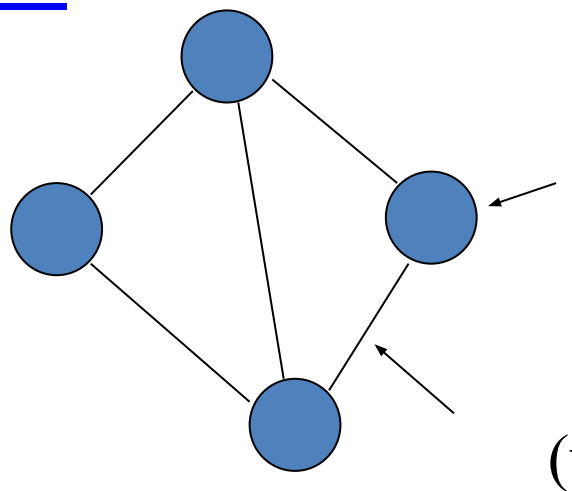


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

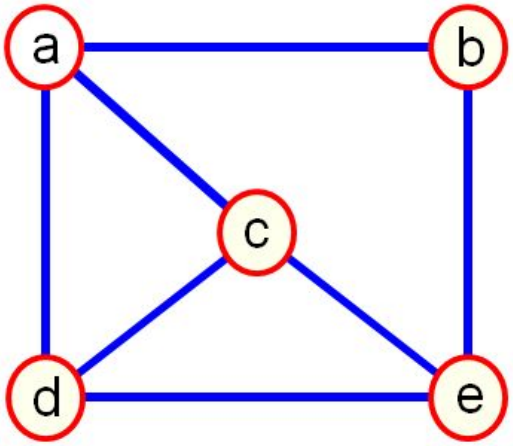
Graph - Introduction

- Definition
  - A **graph** is a **non-linear** and **non-hierarchical** data structure that organizes data elements, called **vertices**, by connecting them with links, called **edges**



Graph - Introduction

- A graph G = (V,E) is composed of:
  - V: set of vertices (nodes)
  - E: set of edges (arcs) connecting the vertices V
- An edge e = (u,v) is a pair of vertices

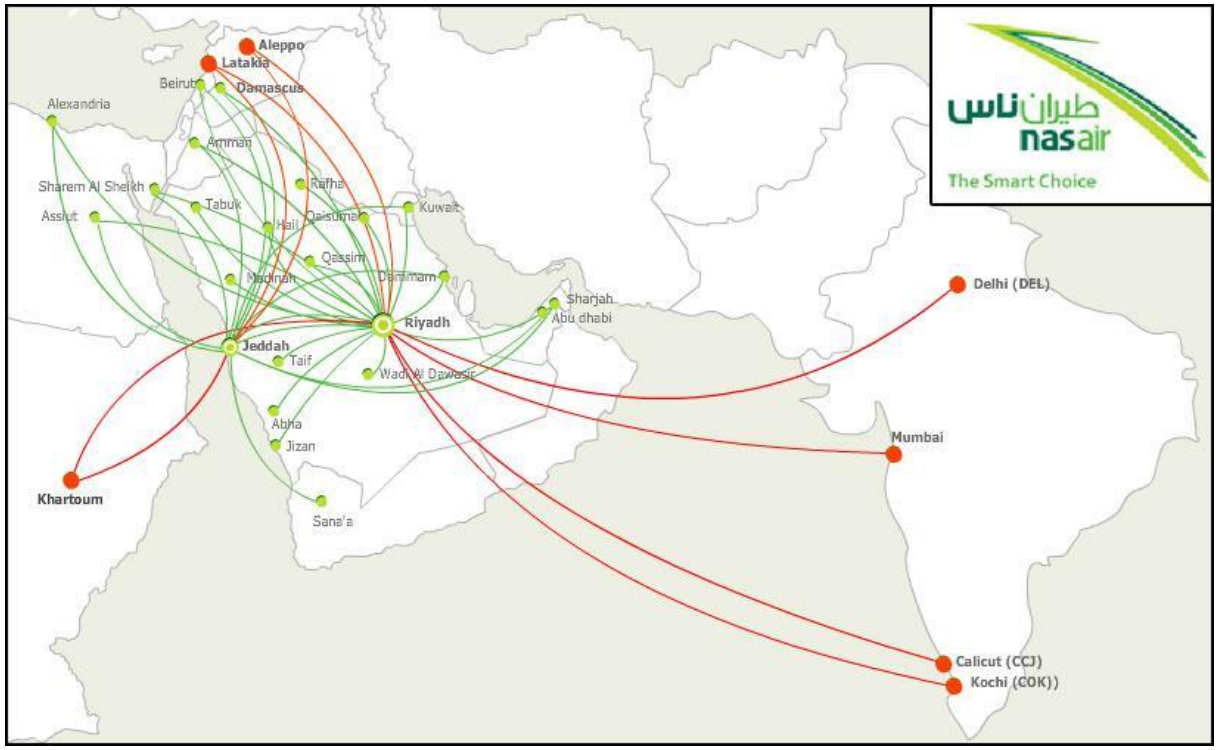


V= {a,b,c,d,e}

E=
{(a,b),(a,c),(a,d),(b,e),(c,d),(c,e),(d,e)}

Graph

Graph Examples - nasair flight network

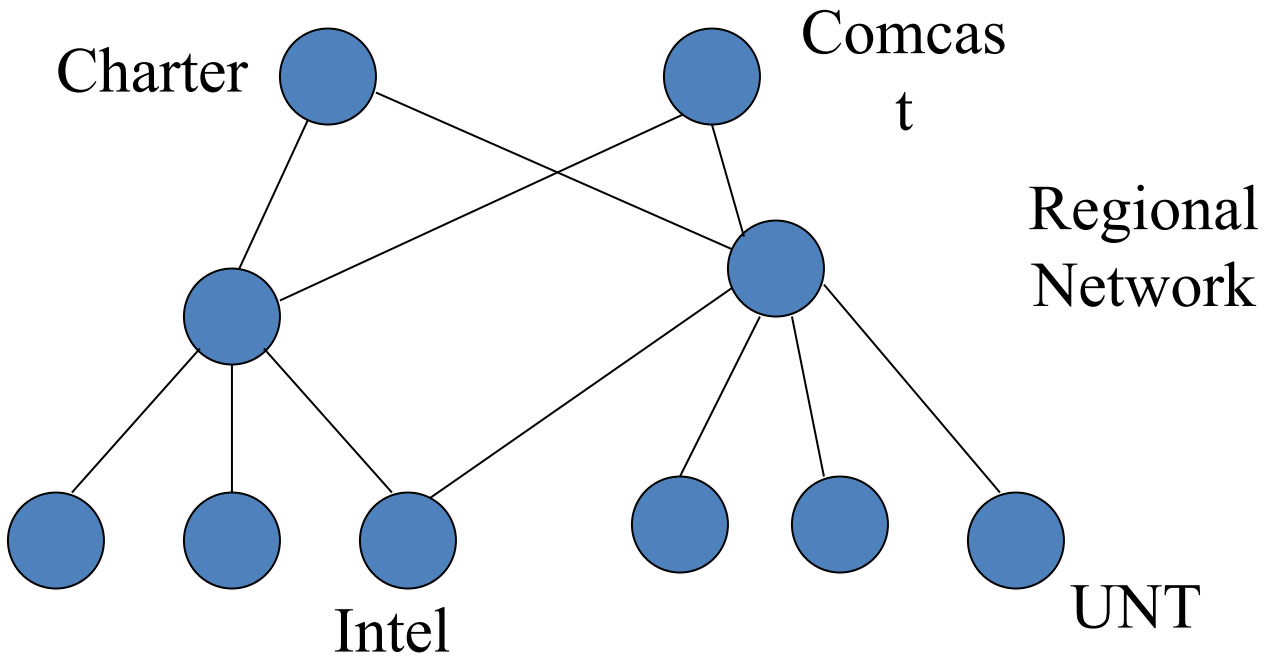


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Graph

Graph Examples – Computer Network

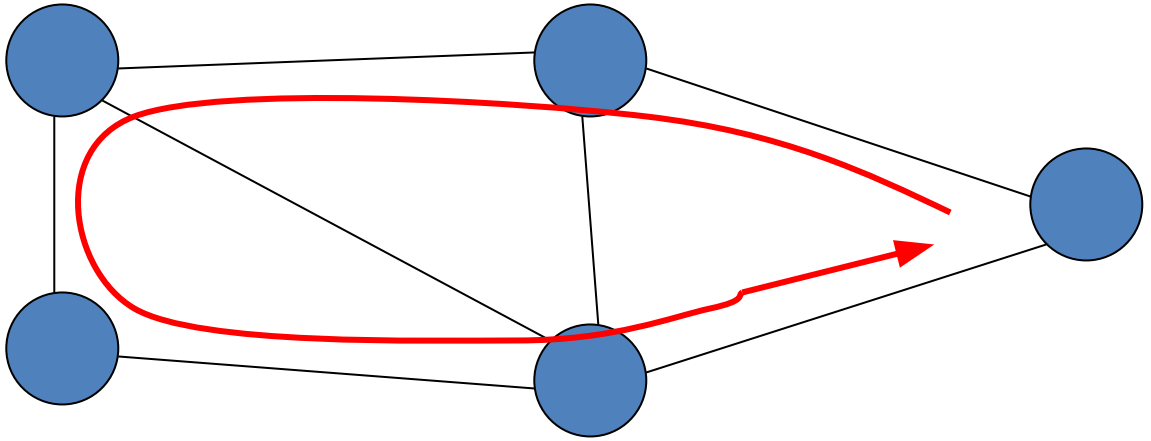


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Graph - Application

- Traveling Salesman

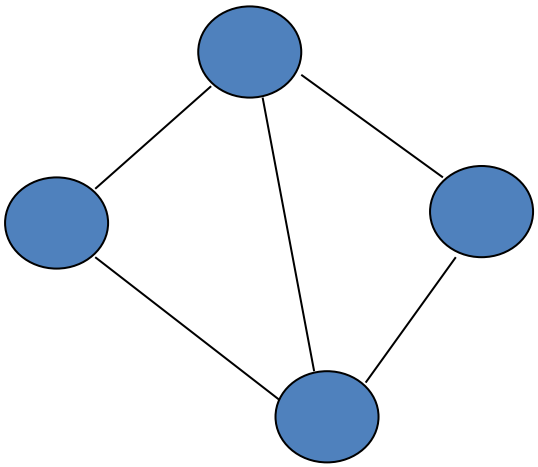


- Find the shortest path that connects all cities without a loop.

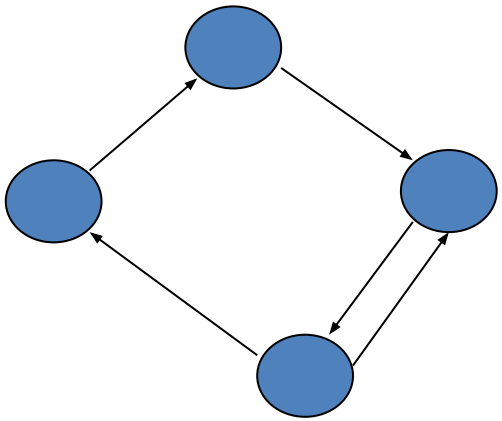
Graph

Graph - Types

- Undirected Graph
  - Edge has no orientation



- Directed Graph
  - Edge has oriented vertex



Graph

Terminologies

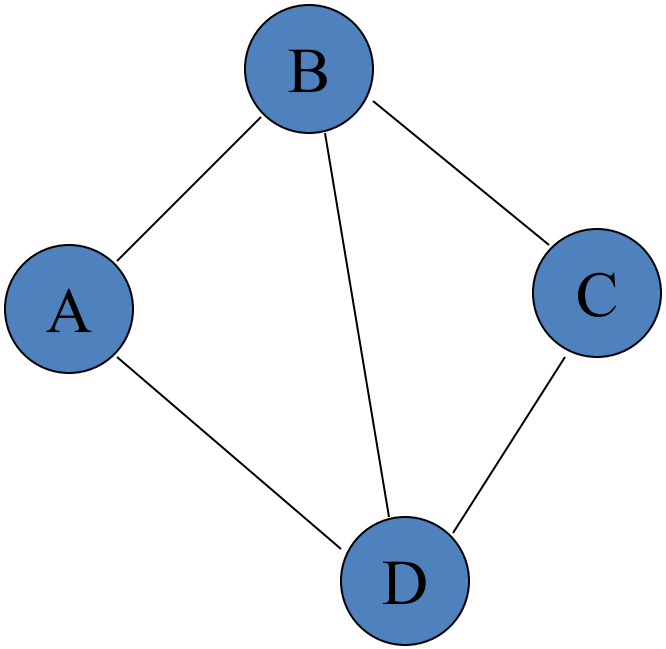
Dr. Jonathan (Yahya) Cazalas

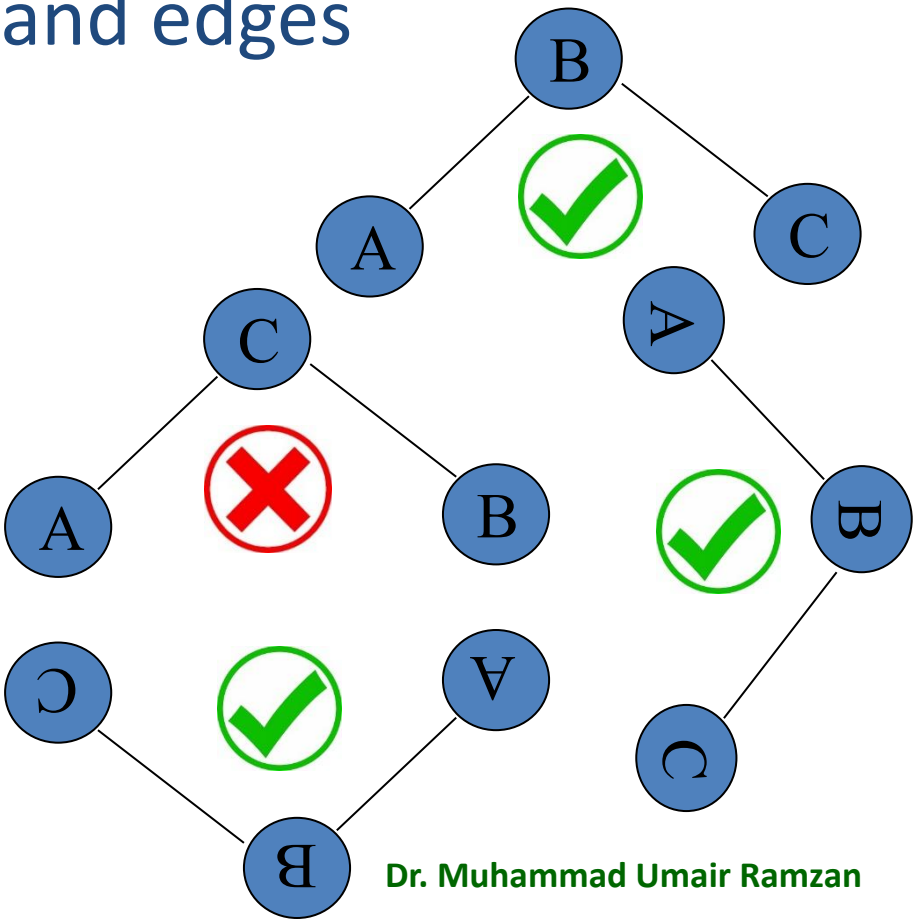
Dr. Muhammad Umair Ramzan

Graph

Terminologies - **Sub graph**

- Subset of vertices and edges



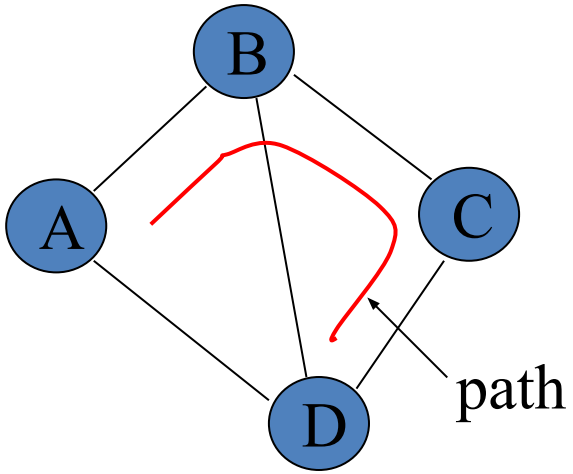


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

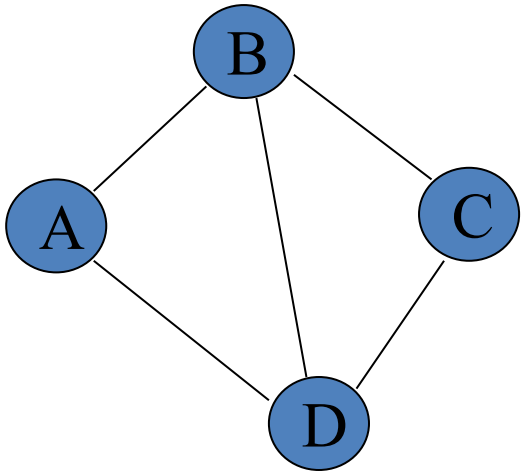
Terminologies –Path

- A path is a walk from one vertex to any other vertex.
- The vertices traversed to reach to a vertex from an other
  - *ABCD* is a path
  - ADC ✓
  - ABDA ✓
  - BCDA ✓
  - ABDCBA ✓
  - ABCA ✗



Terminologies – Simple Path

- A simple path is a path such that all vertices are distinct
  - *ABCD* is a simple path
  - ADC ✓
  - ABDA ✗
  - BCDA ✓
  - ABDCBA ✗
  - ABCA ✗

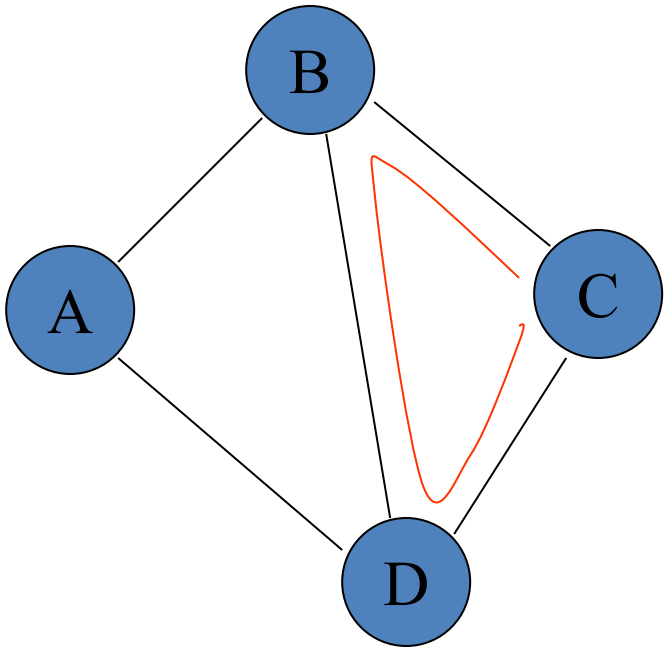


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Terminologies – **Cycle**

- A cycle is a path that starts and ends at the same point. For undirected graph, the edges are distinct.
  - *CBDC* is a Cycle
  - ABCD ✗
  - ABDA ✓
  - BCDB ✓
  - ABDCBA ✓

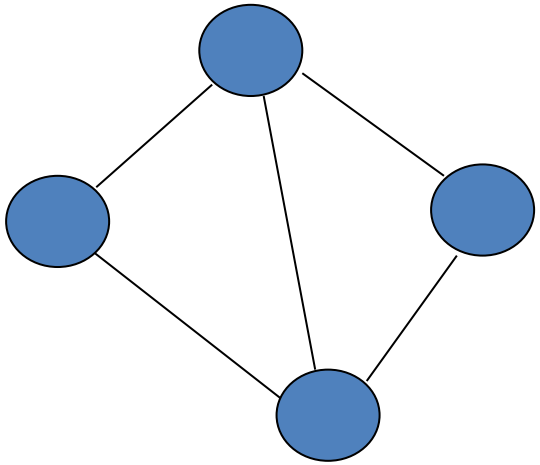


Dr. Jonathan (Yahya) Cazalas

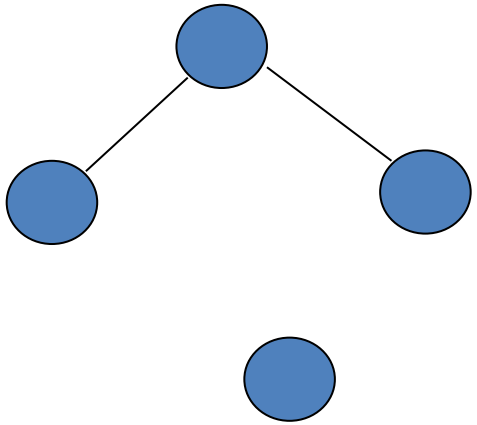
Dr. Muhammad Umair Ramzan

Graph – Connected / Unconnected

- Connected Graph
  - All vertices are connected

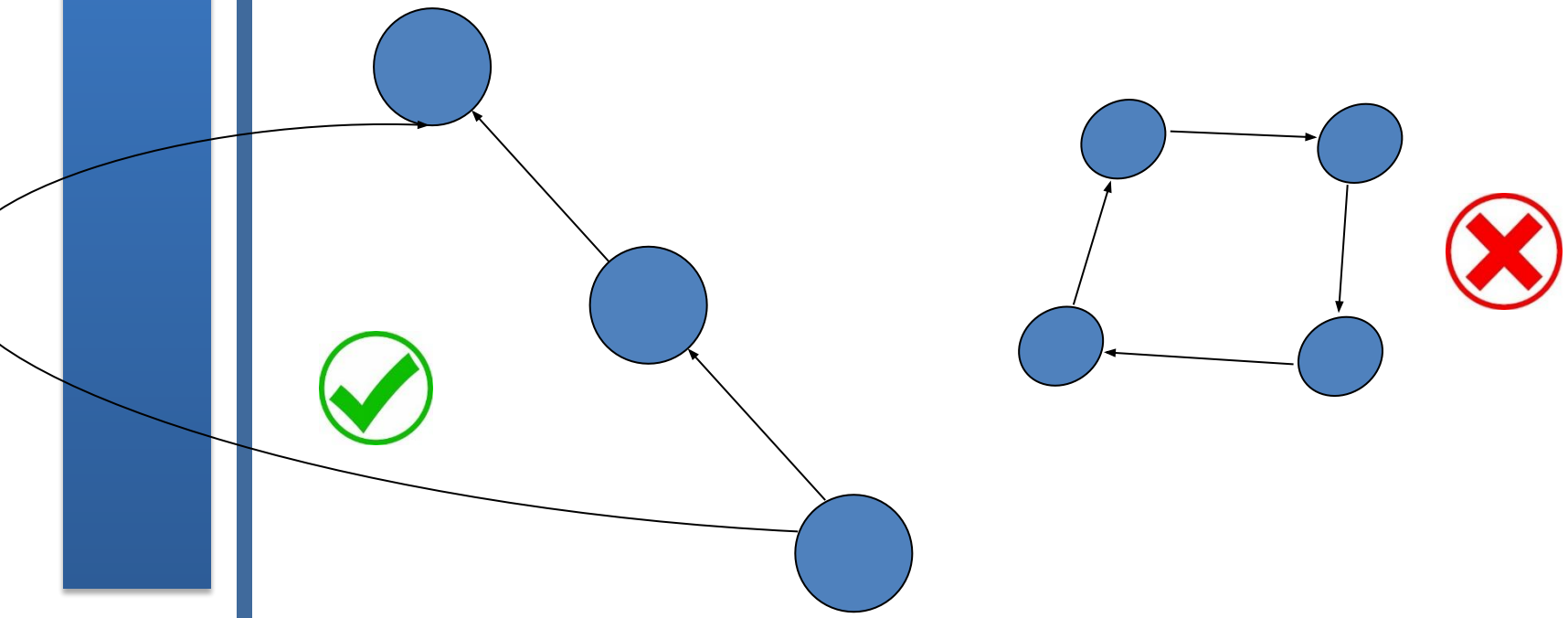


- Unconnected Graph
  - One or more vertices are not linked



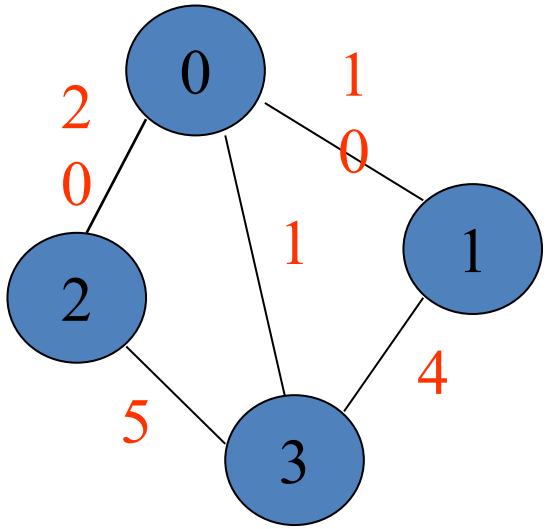
Terminologies – DAG

- Directed Acyclic Graph (DAG) : directed graph without cycle



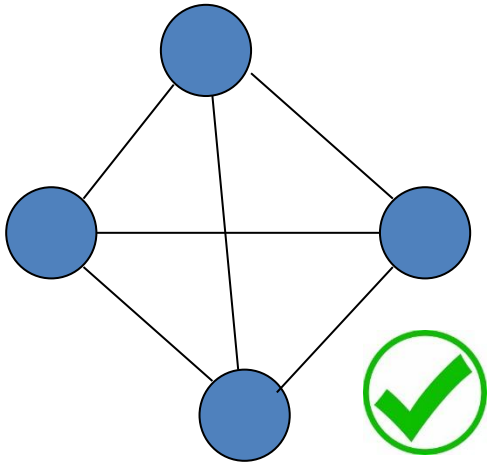
Terminologies – **Weighted Graph**

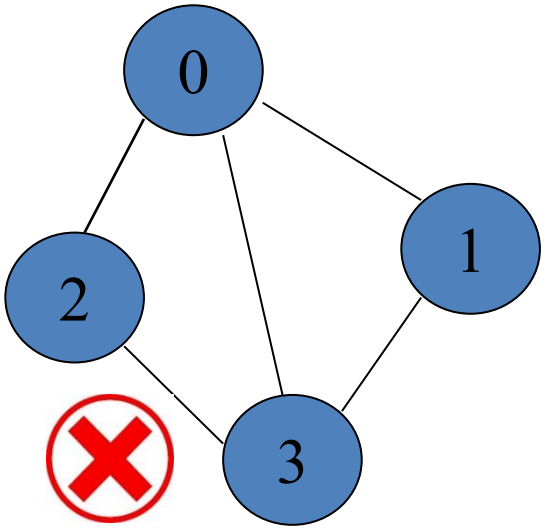
- Weighted graph: a graph with numbers assigned to its edges
  - cost, distance, travel time, etc.



Terminologies – Complete Graph

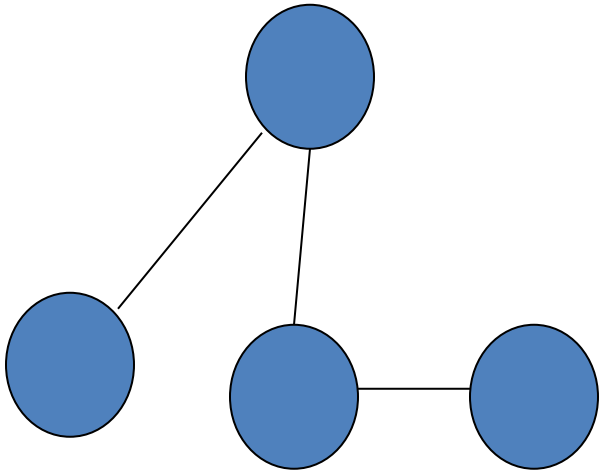
- There is a direct edge between every two vertices
- Total edges $E = N(N-1)/2$





Terminologies – Sparse Graph

- There is a very small number of edges in the graph
- Total edges $E = N-1$



Graph

Representation

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Graph - **Representation**

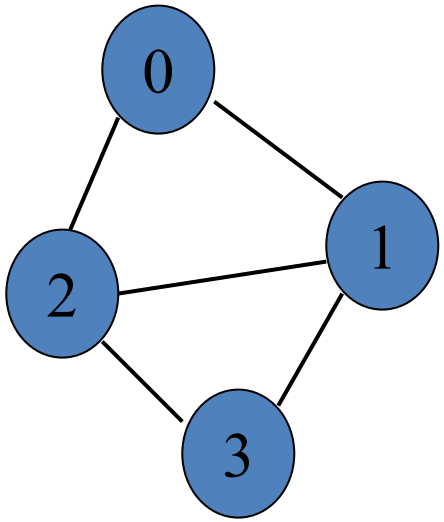
- Adjacency Matrix
- Adjacency List

Representation – Adjacency Matrix

- Assume N nodes in graph
- Use Matrix A[0…N-1][0…N-1]
  - if vertex i and vertex j are adjacent in graph, A[i][j] = 1,
  - otherwise A[i][j] = 0
  - if vertex i has a loop, A[i][i] = 1
  - if vertex i has no loop, A[i][i] = 0

Graph

Representation – Adjacency Matrix



A[i][j]	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

Undirected Graph

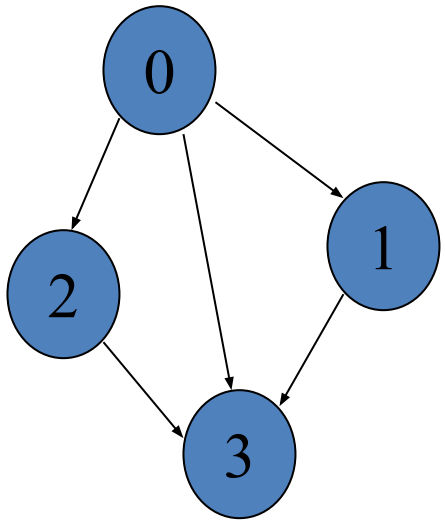
So, Matrix A = $\begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Graph

Representation – Adjacency Matrix



A[i][j]	0	1	2	3
0	0	1	1	1
1	0	0	0	1
2	0	0	0	1
3	0	0	0	0

Directed Graph

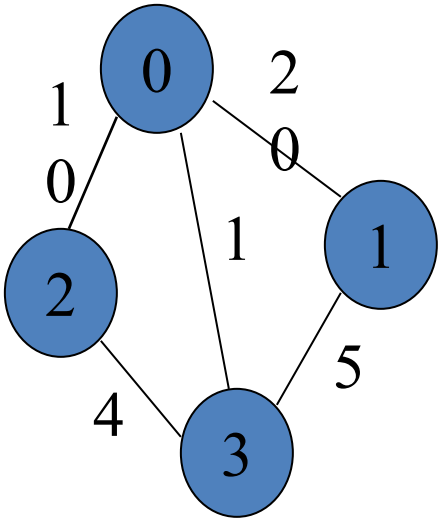
So, Matrix A = $\begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Graph

Representation – Adjacency Matrix



A[i][j]	0	1	2	3
0	0	20	10	1
1	20	0	0	5
2	10	0	0	4
3	1	5	4	0

Weighted Graph

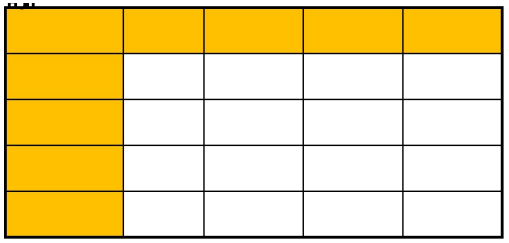
So, Matrix A = $\begin{pmatrix} 0 & 20 & 10 & 1 \\ 20 & 0 & 0 & 5 \\ 10 & 0 & 0 & 4 \\ 1 & 5 & 4 & 0 \end{pmatrix}$

Dr. Jonathan (Yahya) Cazalas

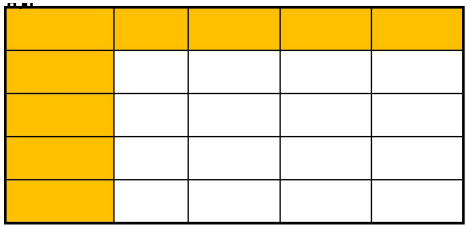
Dr. Muhammad Umair Ramzan

Representation – Adjacency Matrix

- Undirected graph
  - adjacency matrix is symmetric
  - A[i][j]=A[j][i]

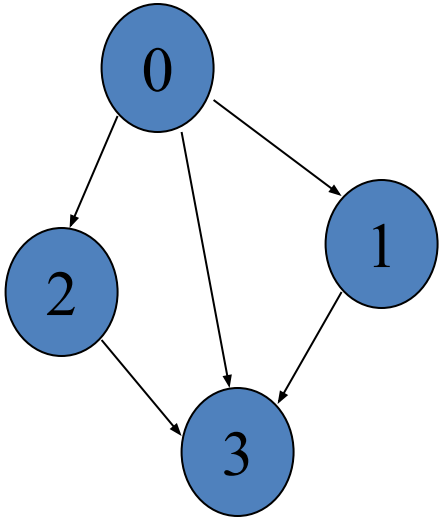


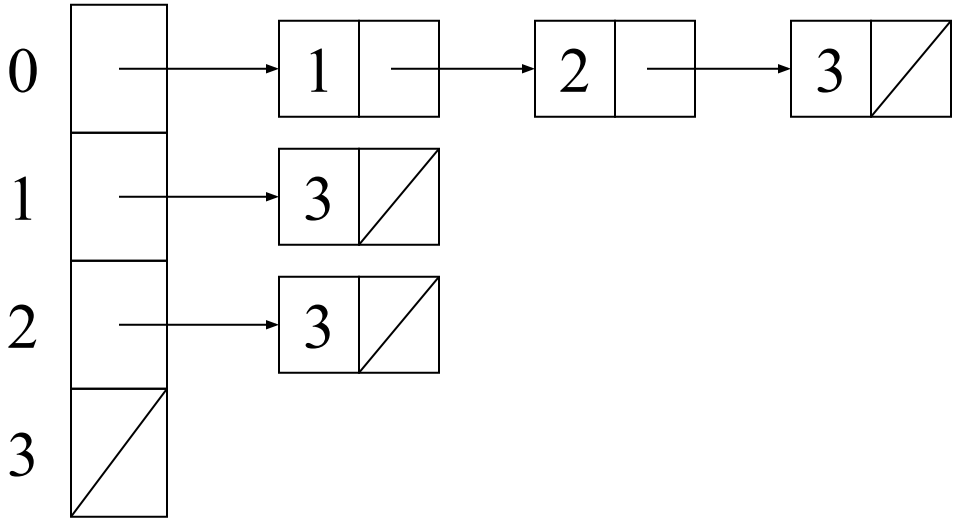
- Directed graph
  - adjacency matrix may not be symmetric
  - A[i][j]≠A[j][i]



Representation – Adjacency List

- An array of lists
- The ***ith*** element of the array is a list of vertices that connect to vertex i

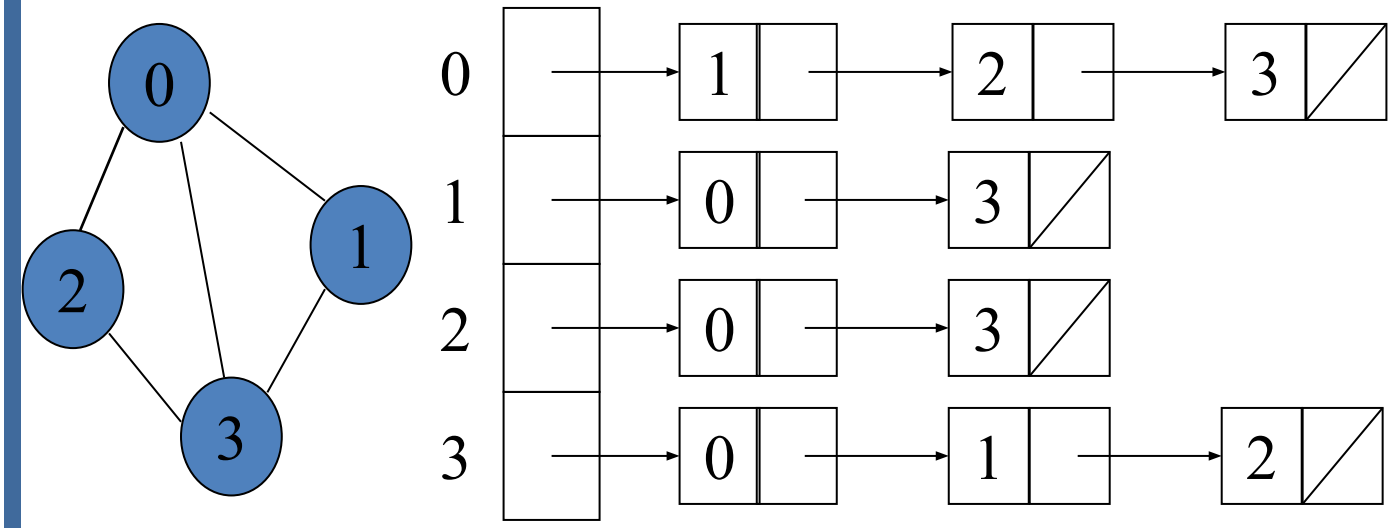




Directed Graph

Representation – **Adjacency List**

- An array of lists
- The ***ith*** element of the array is a list of vertices that connect to vertex i



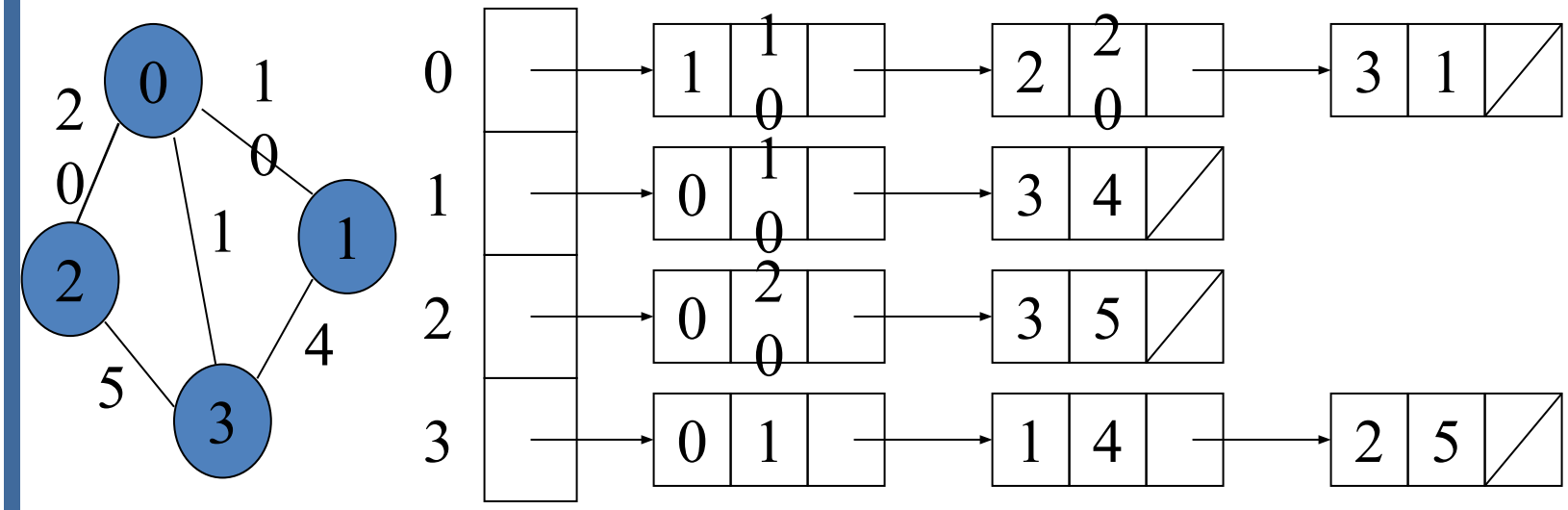
Undirected Graph

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Representation – Adjacency List

- Extend each node with an addition field: weight



Weighted Graph

Representation – Comparisons

Cost	Adjacency Matrix	Adjacency List
Given two vertices u and v: find out whether u and v are adjacent	O(1)	degree of node O(N)
Given a vertex u: enumerate all neighbors of u	O(N)	degree of node O(N)
For all vertices: enumerate all neighbors of each vertex	$O(N^2)$	Summations of all node degree O(E)

Graph

Traversal

Dr. Jonathan (Yahya) Cazalas

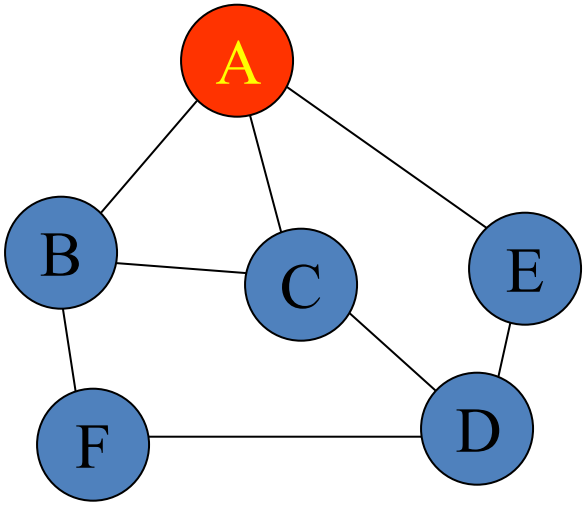
Dr. Muhammad Umair Ramzan

Graph - **Traversal**

- Depth First Search (**DFS**) Traversal
- Breadth First Search (**BFS**) Traversal

Traversal - DFS

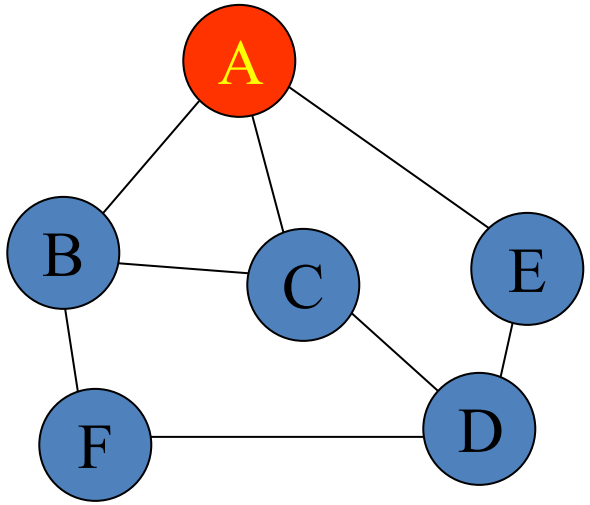
- Probing List is implemented as stack (LIFO)
- Example
  - A's neighbor: B, C, E
  - B's neighbor: A, C, F
  - C's neighbor: A, B, D
  - D's neighbor: E, C, F
  - E's neighbor: A, D
  - F's neighbor: B, D
  - Start from vertex A



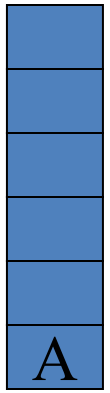
Graph

Traversal - DFS

- A's neighbor: B C E
- B's neighbor: A C F
- C's neighbor: A B D
- D's neighbor: E C F
- E's neighbor: A D
- F's neighbor: B D

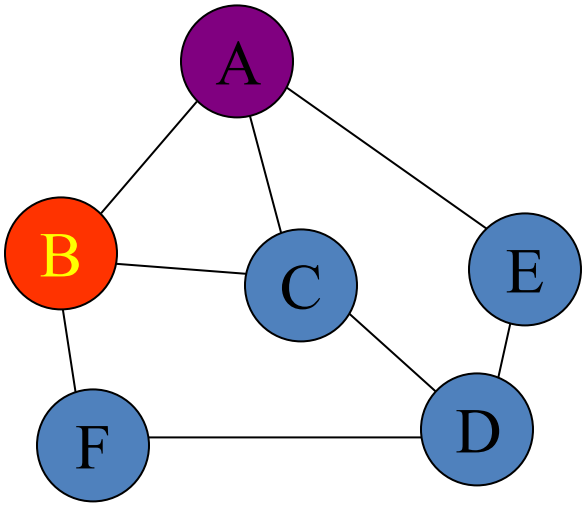


- Initial State
  - Visited Vertices { }
  - Probing Vertices { A }
  - Unvisited Vertices { A, B, C, D, E, F }

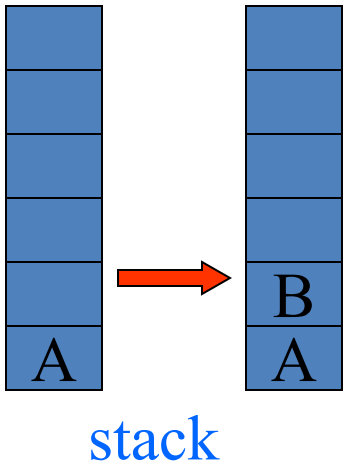


stack

Traversal - DFS

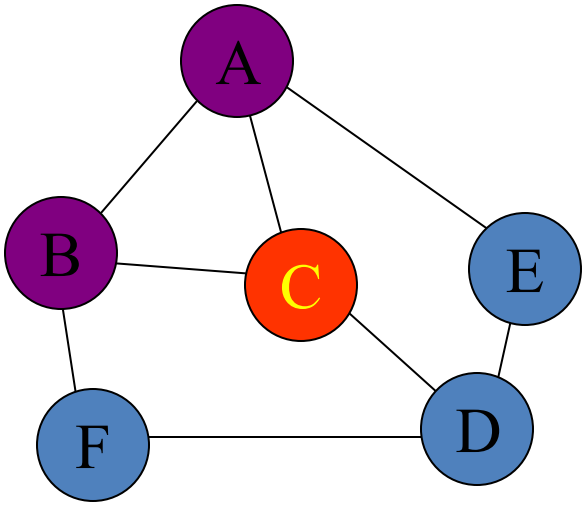


- Pick a vertex from stack, it is A, mark it as visited
- Find A's first unvisited neighbor, push it into stack
  - Visited Vertices { A }
  - Probing vertices { A, B }
  - Unvisited Vertices { B, C, D, E, F }

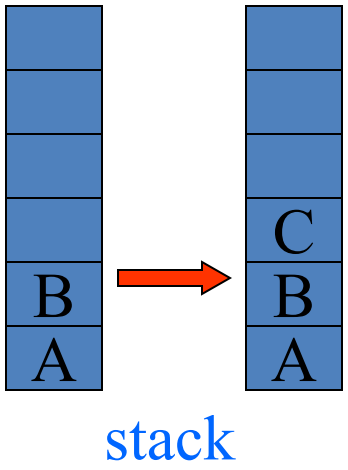


Graph

Traversal - DFS

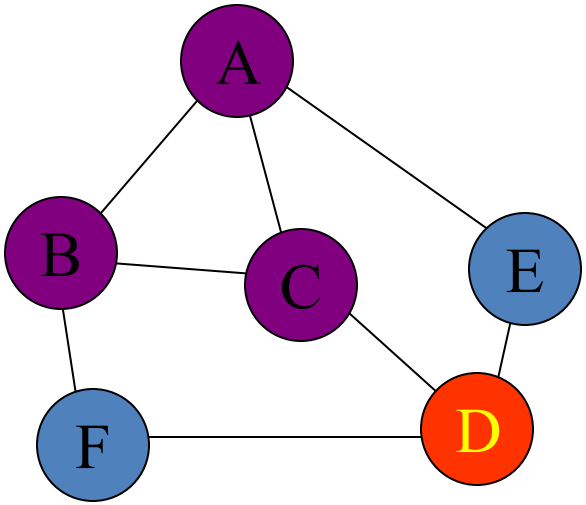


- Pick a vertex from stack, it is B, mark it as visited
- Find B's first unvisited neighbor, push it in stack
  - Visited Vertices { A, B }
  - Probing Vertices { A, B, C }
  - Unvisited Vertices { C, D, E, F }

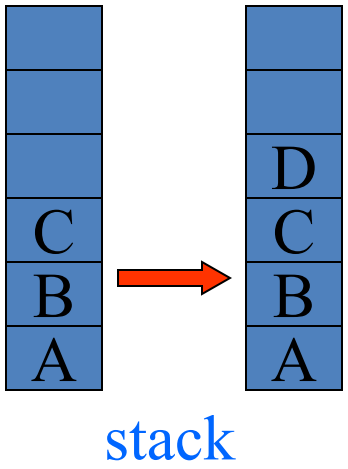


Graph

Traversal - DFS

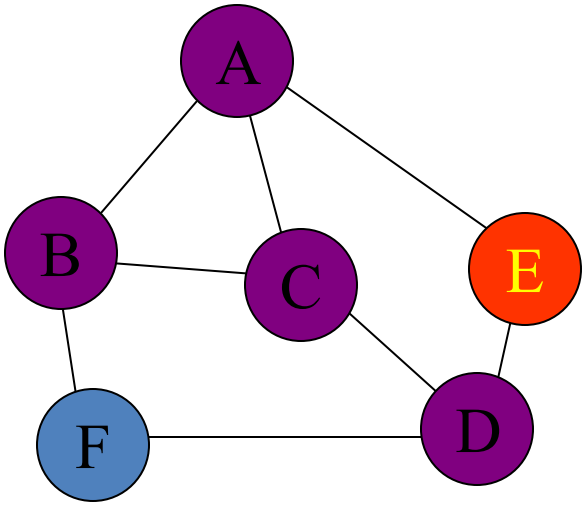


- Pick a vertex from stack, it is C, mark it as visited
- Find C's first unvisited neighbor, push it in stack
  - Visited Vertices { A, B, C }
  - Probing Vertices { A, B, C, D }
  - Unvisited Vertices { D, E, F }

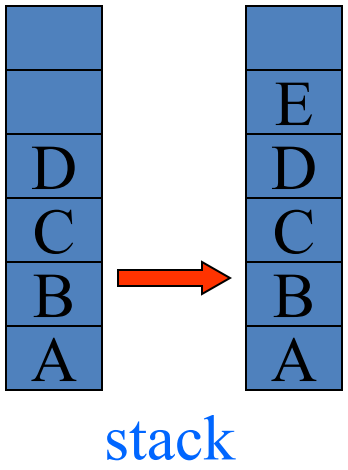


Graph

Traversal - DFS

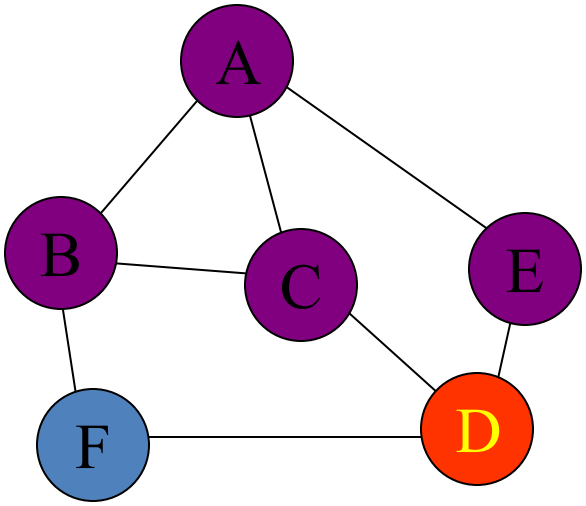


- Pick a vertex from stack, it is D, mark it as visited
- Find D's first unvisited neighbor, push it in stack
  - Visited Vertices { A, B, C, D }
  - Probing Vertices { A, B, C, D, E }
  - Unvisited Vertices { E, F }

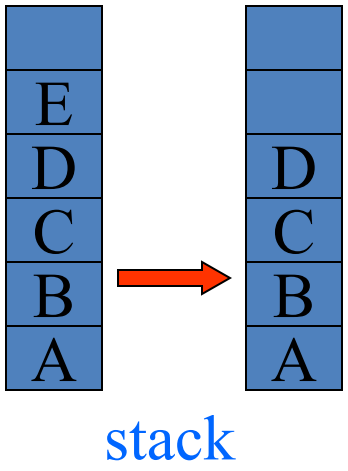


Graph

Traversal - DFS

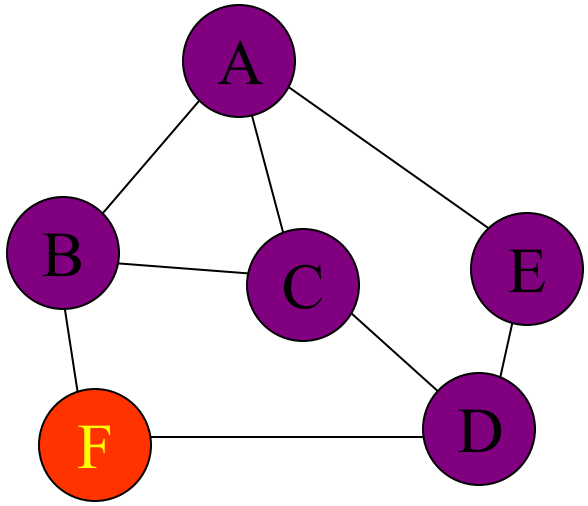


- Pick a vertex from stack, it is E, mark it as visited
- Find E's first unvisited neighbor, no vertex found, Pop E
  - Visited Vertices { A, B, C, D, E }
  - Probing Vertices { A, B, C, D }
  - Unvisited Vertices { F }

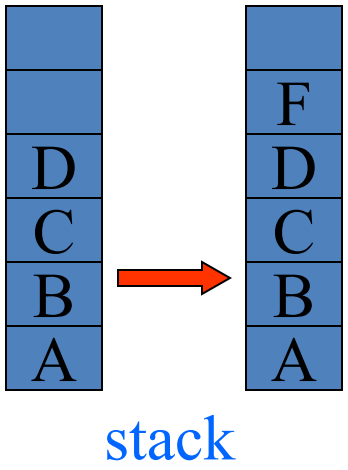


Graph

Traversal - DFS



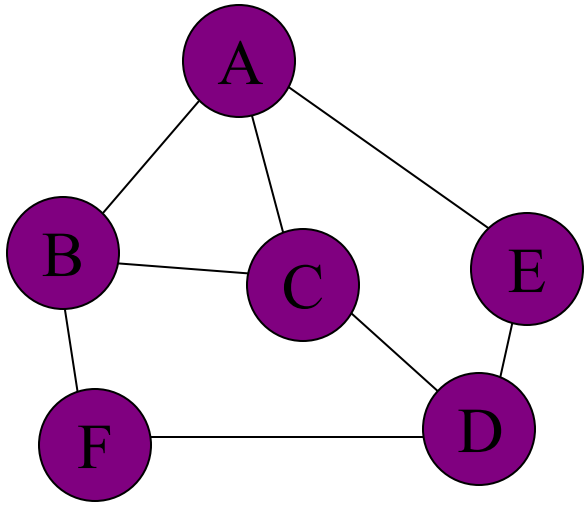
- Pick a vertex from stack, it is D, mark it as visited
- Find D's first unvisited neighbor, push it in stack
  - Visited Vertices { A, B, C, D, E }
  - Probing Vertices { A, B, C, D, F}
  - Unvisited Vertices { F }



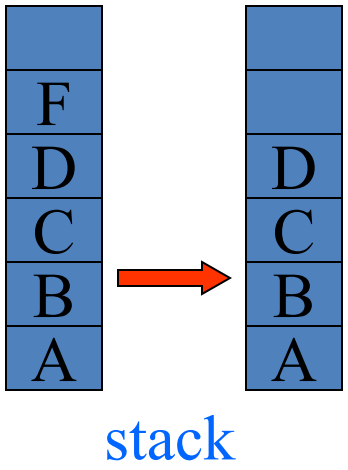
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Traversal - DFS

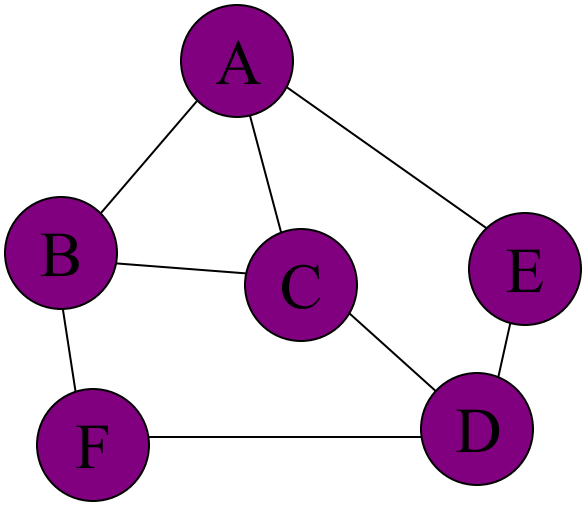


- Pick a vertex from stack, it is F, mark it as visited
- Find F's first unvisited neighbor, no vertex found, Pop F
  - Visited Vertices { A, B, C, D, E, F }
  - Probing Vertices { A, B, C, D}
  - Unvisited Vertices { }

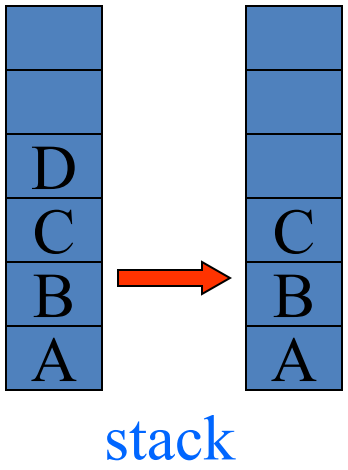


Graph

Traversal - DFS



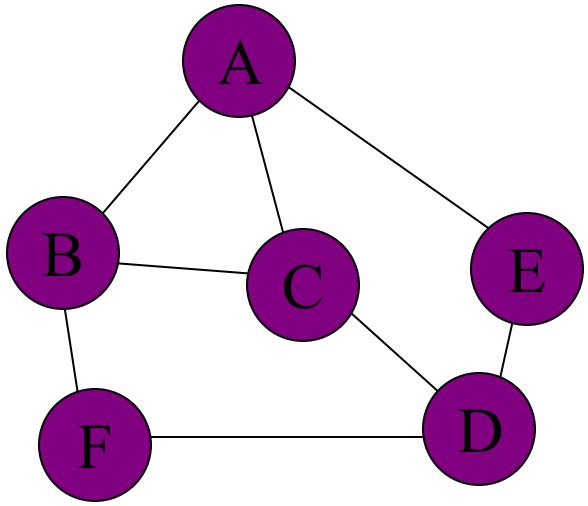
- Pick a vertex from stack, it is D, mark it as visited
- Find D's first unvisited neighbor, no vertex found, Pop D
  - Visited Vertices { A, B, C, D, E, F }
  - Probing Vertices { A, B, C }
  - Unvisited Vertices { }



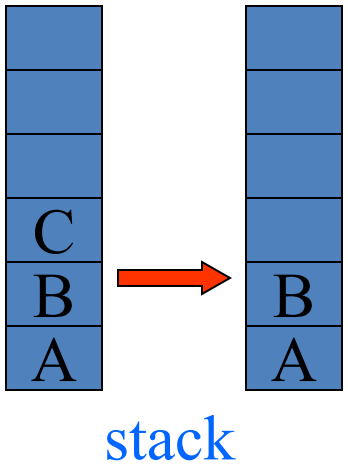
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Traversal - DFS

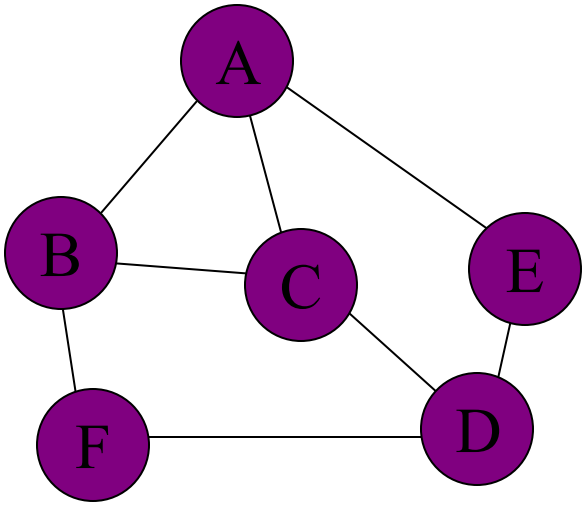


- Pick a vertex from stack, it is C, mark it as visited
- Find C's first unvisited neighbor, no vertex found, Pop C
  - Visited Vertices { A, B, C, D, E, F }
  - Probing Vertices { A, B }
  - Unvisited Vertices { }

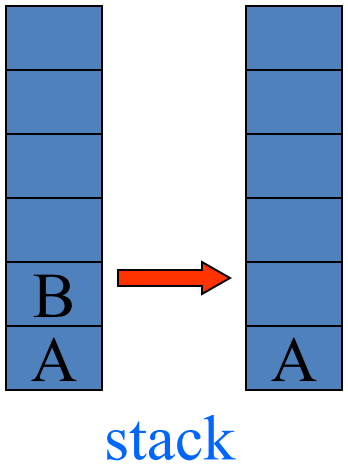


Graph

Traversal - DFS



- Pick a vertex from stack, it is B, mark it as visited
- Find B's first unvisited neighbor, no vertex found, Pop B
  - Visited Vertices { A, B, C, D, E, F }
  - Probing Vertices { A }
  - Unvisited Vertices { }

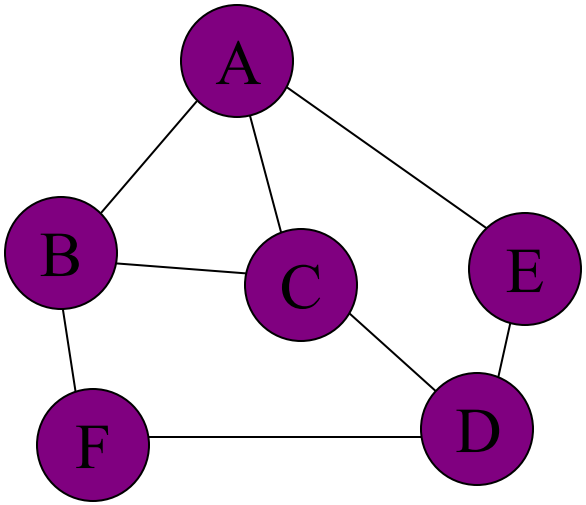


Dr. Jonathan (Yahya) Cazalas

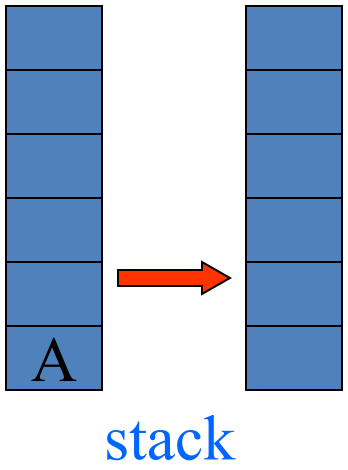
Dr. Muhammad Umair Ramzan

Graph

Traversal - DFS



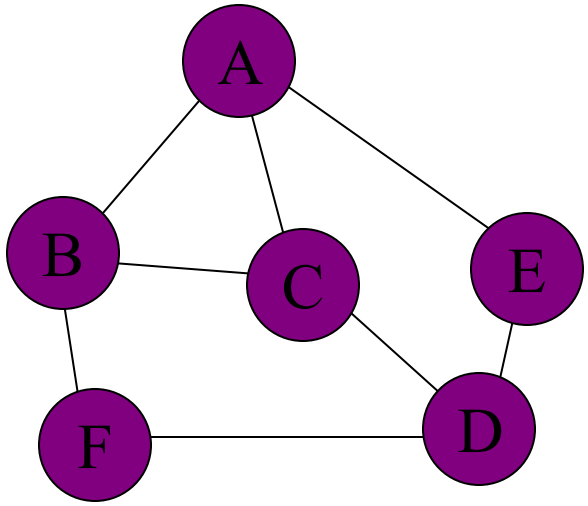
- Pick a vertex from stack, it is A, mark it as visited
- Find A's first unvisited neighbor, no vertex found, Pop A
  - Visited Vertices { A, B, C, D, E, F }
  - Probing Vertices { }
  - Unvisited Vertices { }



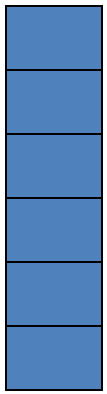
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Traversal - DFS



- Now probing list is empty
- End of Depth First Traversal
  - Visited Vertices { A, B, C, D, E, F }
  - Probing Vertices { }
  - Unvisited Vertices { }



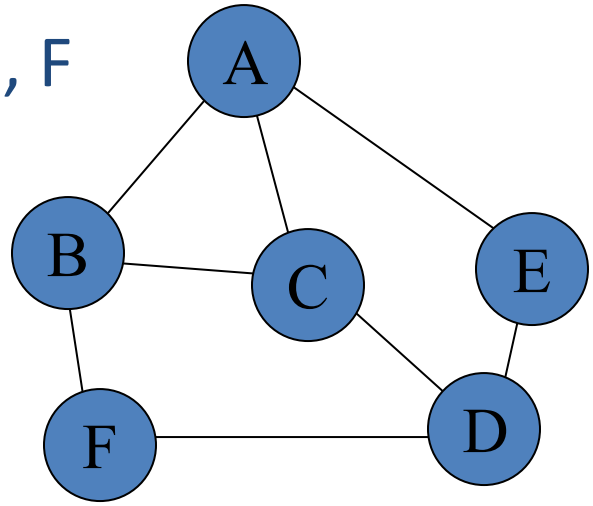
stack

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

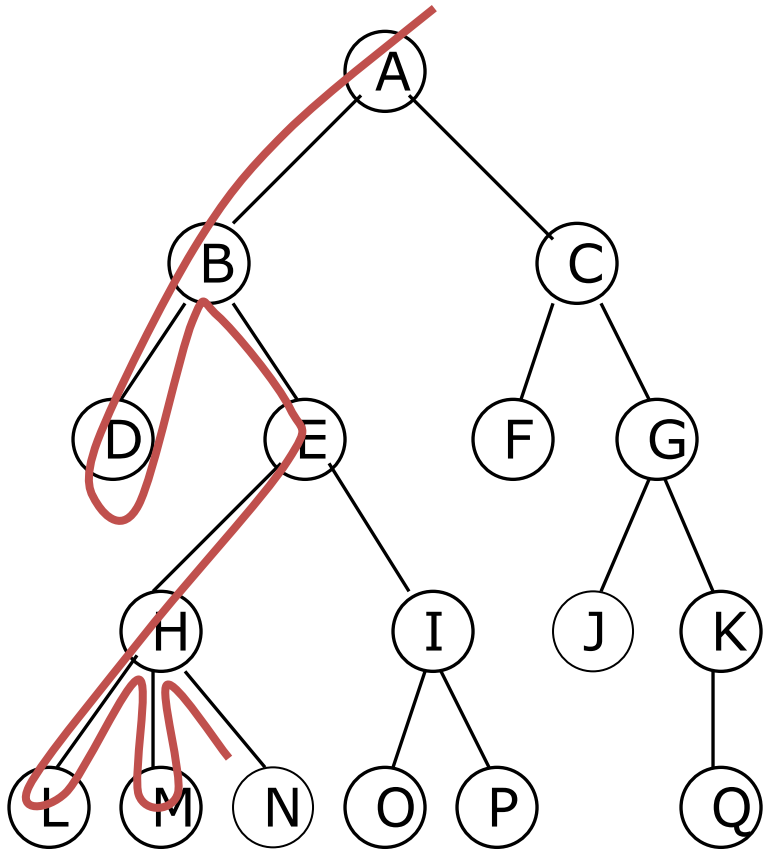
Traversal - **DFS**

- **Depth First Search**(DFS)
  - Order of visited: A, B, C, D, E, F



DFS- Example 02

- **Depth First Search (DFS)**
  - It explores a path **all the way to a leaf** before backtracking and exploring another path



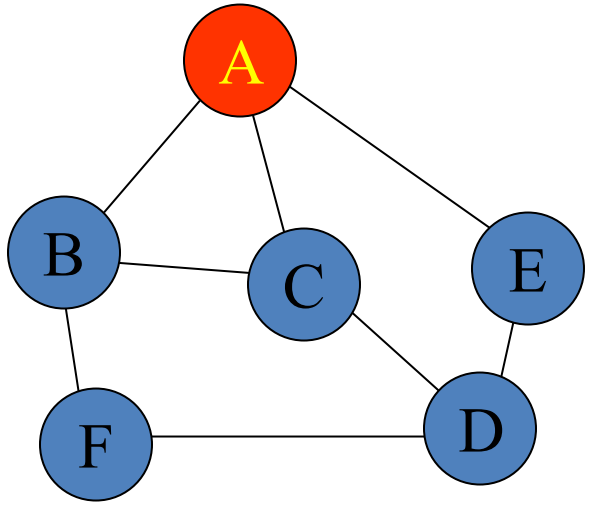
A B D E H L M N I O P C F G J K Q

Dr. Jonathan (Yahya) Cazalas

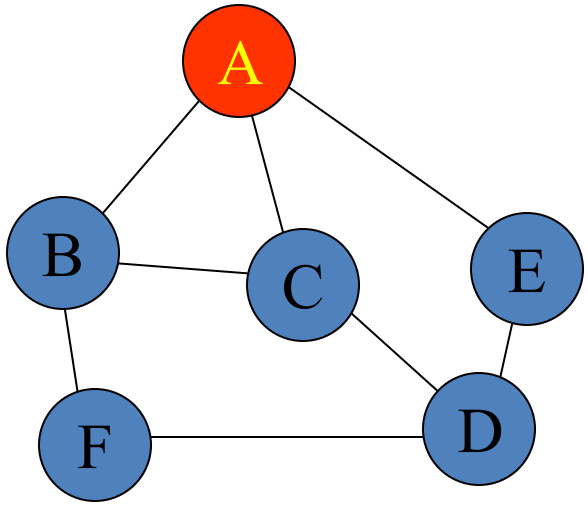
Dr. Muhammad Umair Ramzan

Traversal – **Breadth First Search**

- Probing List is implemented as queue (FIFO)
- Example
  - A's neighbor: B C E
  - B's neighbor: A C F
  - C's neighbor: A B D
  - D's neighbor: E C F
  - E's neighbor: A D
  - F's neighbor: B D
  - Start from vertex A



Traversal – **BFS**



- Initial State
  - Visited Vertices { }
  - Probing Vertices { A }
  - Unvisited Vertices { A, B, C, D, E, F }

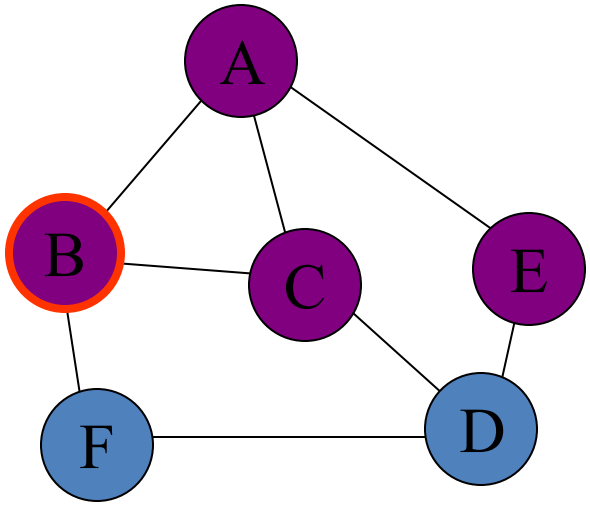
queue

Dr. Jonathan (Yahya) Cazalas

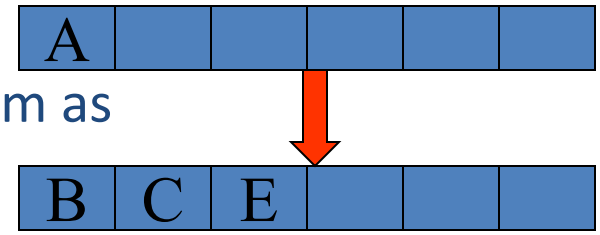
Dr. Muhammad Umair Ramzan

Gr
ap
h

Traversal – BFS



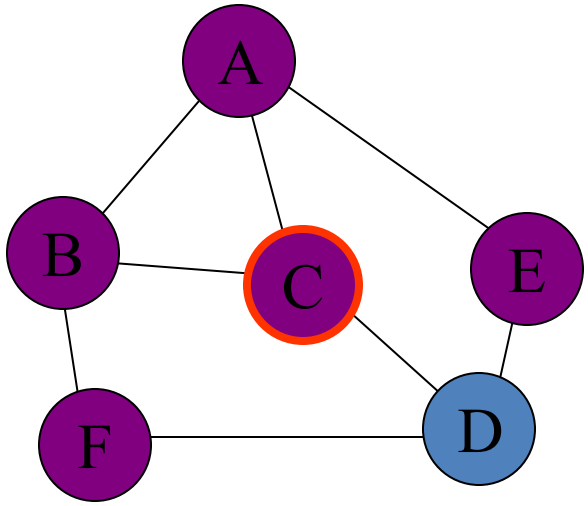
- Delete first vertex from queue, it is A, mark it as visited
- Find A's all unvisited neighbors, mark them as visited, put them into queue
  - Visited Vertices { A, B, C, E }
  - Probing Vertices { B, C, E }
  - Unvisited Vertices { D, F }



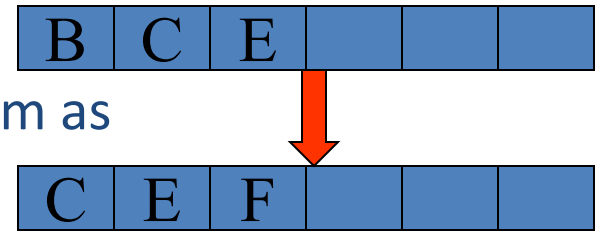
queue

Graph

Traversal – BFS

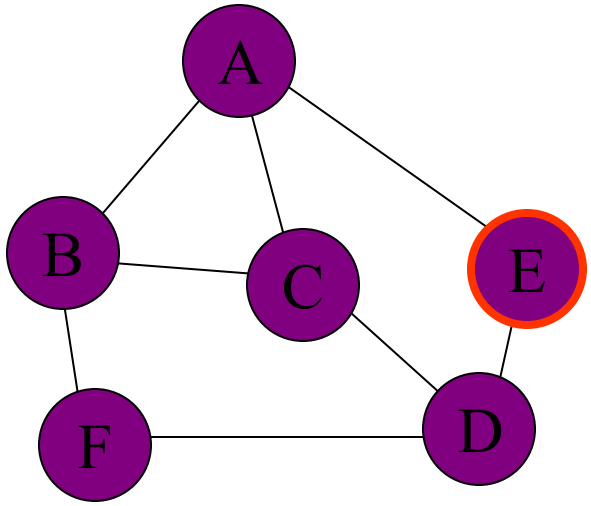


- Delete first vertex from queue, it is B, mark it as visited
- Find B's all unvisited neighbors, mark them as visited, put them into queue
  - Visited Vertices { A, B, C, E, F }
  - Probing Vertices { C, E, F }
  - Unvisited Vertices { D }

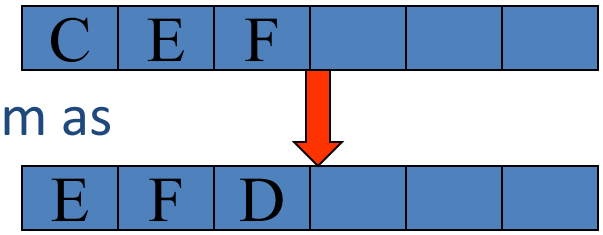


queue

Traversal – BFS



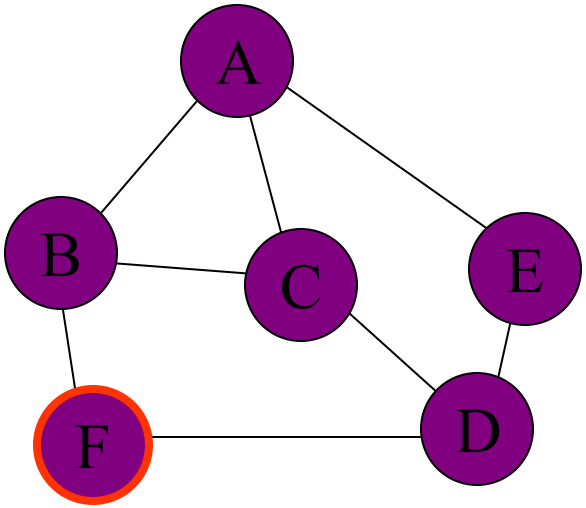
- Delete first vertex from queue, it is C, mark it as visited
- Find C's all unvisited neighbors, mark them as visited, put them into queue
  - Visited Vertices { A, B, C, E, F, D }
  - Probing Vertices { E, F, D }
  - Unvisited Vertices { }



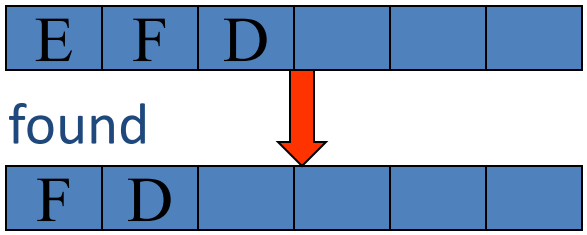
queue

Gr
ap
h

Traversal – BFS



- Delete first vertex from queue, it is E, mark it as visited
- Find E's all unvisited neighbors, no vertex found
  - Visited Vertices { A, B, C, E, F, D }
  - Probing Vertices { F, D }
  - Unvisited Vertices { }



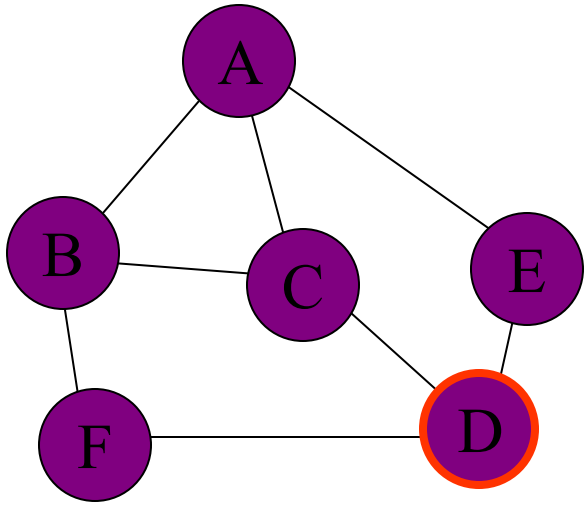
queue

Dr. Jonathan (Yahya) Cazalas

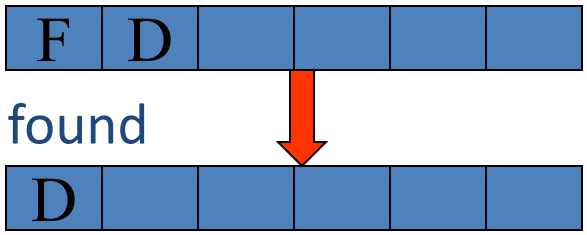
Dr. Muhammad Umair Ramzan

Gr
ap
h

Traversal – BFS



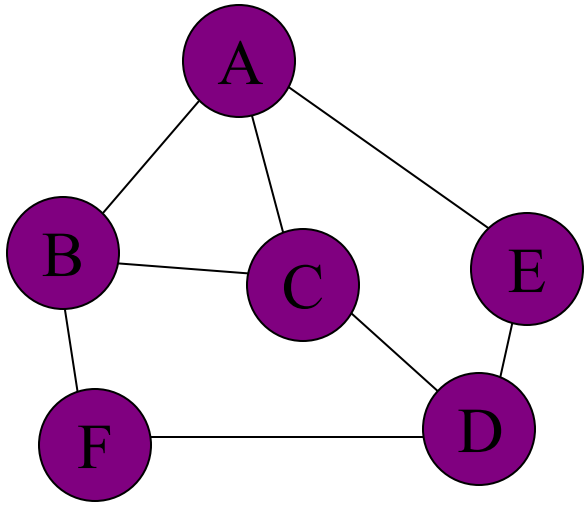
- Delete first vertex from queue, it is F, mark it as visited
- Find F's all unvisited neighbors, no vertex found
  - Visited Vertices { A, B, C, E, F, D }
  - Probing Vertices { D }
  - Unvisited Vertices { }



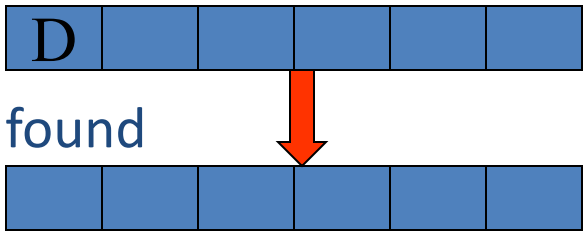
queue

Gr
ap
h

Traversal – BFS



- Delete first vertex from queue, it is D, mark it as visited
- Find D's all unvisited neighbors, no vertex found
  - Visited Vertices { A, B, C, E, F, D }
  - Probing Vertices { }
  - Unvisited Vertices { }

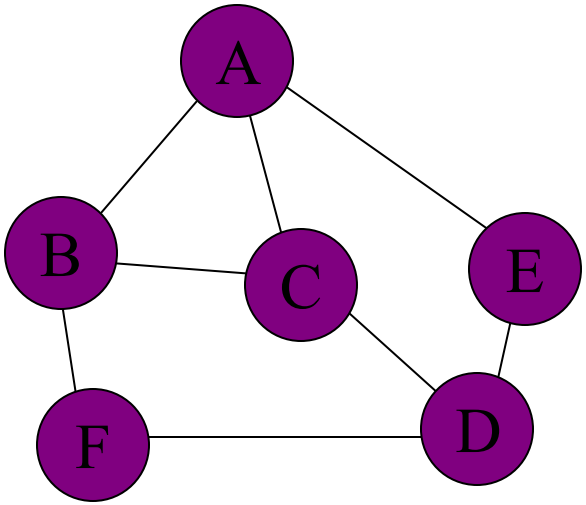


queue

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Traversal – **BFS**



- Now the queue is empty
- End of Breadth First Traversal
  - Visited Vertices { A, B, C, E, F, D }
  - Probing Vertices { }
  - Unvisited Vertices { }

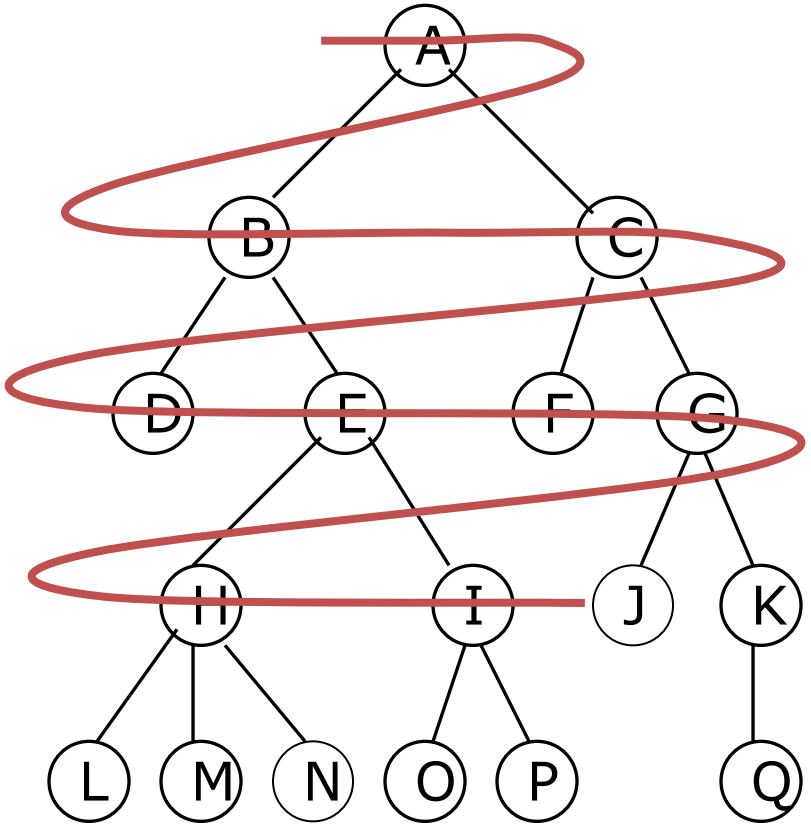
queue

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

BFS- Example 02

- **Breadth First Search (BFS)**
  - It explores **all nodes nearest the root** before exploring nodes further away



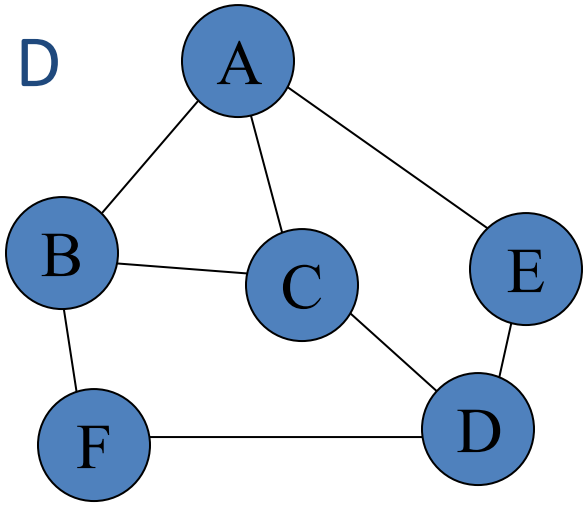
A B C D E F G H I J K L M N O P Q

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

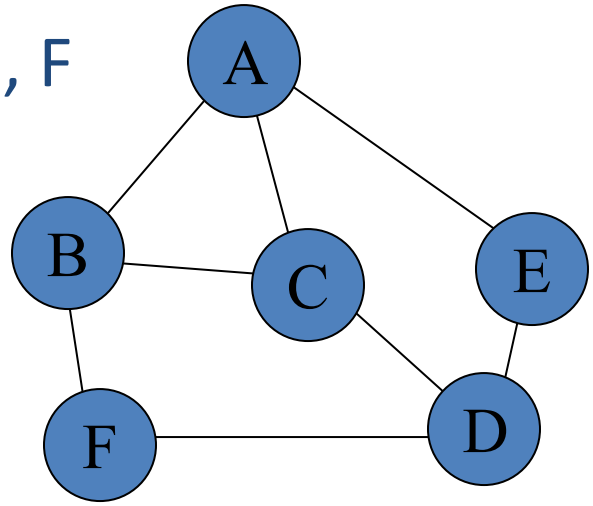
Traversal - **BFS**

- **Breadth First Search** (BFS)
  - Order of visited: A, B, C, E, F, D



Difference – DFS & BFS

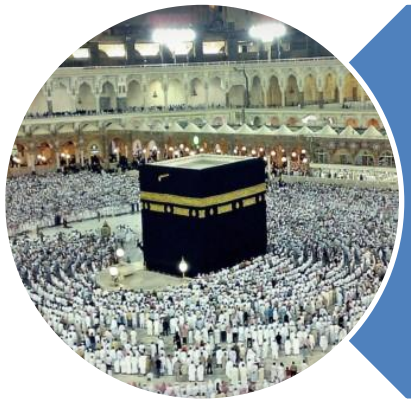
- **Depth First Search**(DFS)
  - Order of visited: A, B, C, D, E, F



- **Breadth First Search** (BFS)
  - Order of visited: A, B, C, E, F, D

Graph Material

Gr
ap
h



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).**(Quran 3 : 173)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan